Memory Task Explored through Energy Efficiency in Biological and Artificial Neural Networks

Pearl Oyewole

California Institute of Technology
poyewole@caltech.edu

Google Colab Link

Abstract

This project investigates how different neural memory mechanisms scale with increasing memory demands using the copy task as a delayed recall benchmark. Two architectures are compared: a Gated Recurrent Unit (GRU), representing an artificial neural network with recurrent hidden-state memory, and a Hebbian-style model, representing a biologically inspired baseline with synaptic-style memory storage. By analyzing how energy consumption changes with delay length, this project explores how different memory mechanisms trade off efficiency and performance.

1 Introduction

Energy consumption is a major challenge in modern computing, especially with the rapid rise of large language model usage among individuals and corporations. One of the largest contributors to this trend is the electricity required to operate artificial intelligence data centers. According to the World Resources Institute, institutional projections estimate that generative AI data center electricity demand could range from about 200 TWh to over 1,000 TWh annually by 2030. Separately, data-center power demand in North America was already in the multi-gigawatt range by the end of 2022. Although computing hardware has advanced to support this growing load, biological neural networks (BNNs) in living organisms still greatly outperform most artificial neural networks (ANNs) in terms of energy efficiency.

Brain adaptations arise from evolutionary pressures, environmental conditions, and specific ecological niches. For example, the human brain is estimated to use about 12 to 20 watts of power during normal operation. This is remarkably low compared with many engineered computing systems during active use. Much of this difference in energy cost is related to how information is stored, updated, and reused. The human brain consists of distributed networks of strongly and reciprocally connected neurons whose synapses change through experience. One important principle is Hebbian learning, often summarized as "neurons that fire together, wire together." Hebbian learning helps explain how synaptic connections strengthen through repeated co-activation. In contrast, many artificial neural networks are trained using gradient-based optimization. These methods can be used in supervised or unsupervised settings, although many practical ANN systems are trained with labeled data. Hebbian-style models instead emphasize local learning, where synaptic connectivity changes based on the simultaneous activation of pre- and post-synaptic neurons.

By examining the energy cost of simulating different memory mechanisms, and by comparing hardware-level estimates with operation-based analytical estimates, we can better understand how different neural architectures behave as energy-constrained systems. The goal of this project is to compare how energy consumption changes with memory length for different memory mechanisms and to relate these scaling patterns to biological ideas about memory.

2 Materials

All experiments were run in Google Colab using PyTorch. A GPU in Google Colab was used to speed up the training process. I also used NumPy, pandas, matplotlib, and CodeCarbon for estimating electricity use and emissions during model training and evaluation.

To keep results consistent, I fixed a random

seed across Python, NumPy, and PyTorch. This seed variable was set to 1024. This helps reproducibility and gives more consistent outcomes across runtimes. The copy task used sequences of length 5 and delays of 1, 5, 10, 20, and 40 time steps. The GRU used a hidden size of 64 and was trained for 300 epochs using the Adam optimizer.

2.1 Copy Task

In the copy task, the input consists of:

- A sequence of random symbols
- A delay period with blank inputs
- A special cue token indicating recall

The model must output blank tokens during the delay and then reproduce the original sequence at the end once given the cue.

2.2 GRU Model

The GRU model consists of:

- An embedding layer
- A GRU layer
- A linear output layer

The model is trained using cross-entropy loss across all time steps. Performance is evaluated using recall accuracy, which only considers the final output positions where the stored sequence must be reproduced.

2.3 Hebbian Model

The Hebbian model is a simple, non-learned baseline. It stores the input sequence in a memory trace and applies decay during the delay period. At recall time, it outputs the stored sequence. This model does not use gradient descent and is designed to simulate a basic form of synaptic-style memory. Real biological memory is much more complex than this simplified baseline, but this model is still a useful conceptual example of local learning and local information storage in neural systems.

2.4 Energy Measurement

Two methods to estimate energy were used:

1. **CodeCarbon**: Estimates electricity usage during model training and evaluation based on hardware usage.
2. **Analytical Estimates**: Estimates energy based on operation counts (e.g., multiply-accumulate operations) using values inspired by hardware literature (Horowitz).

These two methods are not directly comparable, but together they help analyze different aspects of energy use.

3 Results

Recall accuracy showed the clearest dependence on delay. At delay 1, the GRU achieved perfect recall (1.00 accuracy). However, as the delay increased, performance dropped steadily. At delay 5, recall accuracy was about 0.59, at delay 10 it was about 0.35, at delay 20 it fell to about 0.21, and at the longest delay of 40 it dropped to about 0.12. In contrast, the Hebbian model maintained perfect recall accuracy (1.0) across all delay lengths. This means that in this task, the Hebbian baseline was not affected by delay in the same way as the GRU.

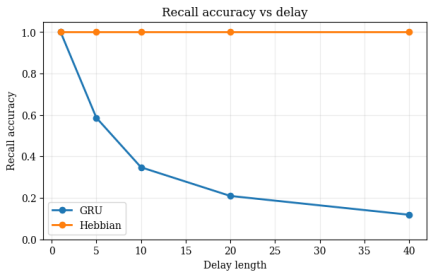


Figure 1: Recall accuracy as a function of delay length. GRU performance declines strongly with longer delays, while the Hebbian model remains at perfect recall on this task.

The CodeCarbon estimates showed that GRU training consistently used the most electricity, followed by Hebbian evaluation, with GRU evaluation being the lowest. GRU training energy stayed in the range of roughly 1.4×10^{-5} to 1.8×10^{-5} kWh across delays. GRU evaluation energy was much smaller, rising from about 5.2×10^{-7} kWh at delay 1 to about 1.5×10^{-6} kWh at delay 40. Hebbian evaluation energy ranged from about 6.5×10^{-6} to 1.1×10^{-5} kWh, which was higher than GRU evaluation at every delay and, at some delays, surprisingly close to GRU training.

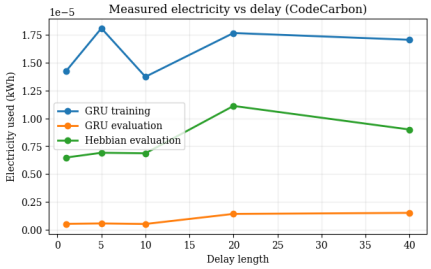


Figure 2: CodeCarbon-estimated electricity usage as a function of delay length. GRU training has the highest estimated electricity use, while Hebbian evaluation is consistently higher than GRU evaluation.

The analytical energy estimates showed a cleaner scaling pattern for the GRU. GRU analytical energy increased approximately linearly with delay, from about 8.0×10^{-7} J at delay 1 to about 5.3×10^{-6} J at delay 40. In contrast, the Hebbian analytical estimate remained much smaller, increasing from about 1.2×10^{-10} J to about 1.68×10^{-9} J across the same delays. On this operation-based scale, the Hebbian model stayed far below the GRU.

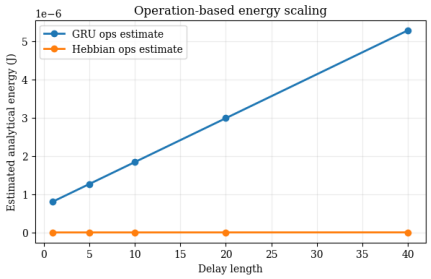


Figure 3: Operation-based analytical energy scaling. The GRU estimate increases strongly with delay, while the Hebbian estimate remains much smaller across all delays.

Evaluation runtime helps explain some of the difference between the measured and analytical results. The GRU evaluation runtime stayed around 0.04–0.05 seconds across delays, while the Hebbian evaluation runtime ranged from about 0.32 to 0.55 seconds. So even though the Hebbian model had a very small operation-based energy estimate, its actual execution in this implementation took longer and therefore showed a higher estimated hardware-side cost.

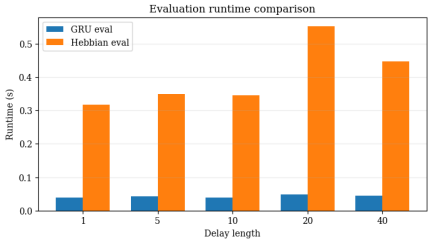


Figure 4: Evaluation runtime comparison. The Hebbian implementation takes substantially longer to evaluate than the GRU, which helps explain why its CodeCarbon estimates are higher than its analytical estimates would suggest.

Overall, these results show that the GRU and Hebbian models have very different scaling behavior. The GRU shows a clear tradeoff between memory length, accuracy, and energy efficiency, while the Hebbian model avoids much of this tradeoff on the copy task itself but still incurs a noticeable measured runtime and hardware-side cost in this implementation.

4 Discussion

The main result of this project is that the type of memory mechanism strongly affects both performance and energy behavior. The GRU model had to learn how to store information in its hidden state, and recall became much less reliable as the delay increased. The Hebbian model performed perfectly on all delays, but this is partly because it was built specifically for this task. It stores information more directly rather than learning a general strategy. Because of this, it is not a completely fair comparison, but it still shows how a different type of memory system can behave.

One interesting part of the results is that training loss and recall accuracy did not always tell the exact same story. Some longer-delay GRU runs still reached fairly low cross-entropy loss, even when recall accuracy was poor. That likely happens because the loss is averaged across all time steps, including many blank outputs that are easier to predict, while recall accuracy only focuses on the final positions that actually test memory. So in this setting, recall accuracy ends up being the more meaningful metric for the memory question.

The energy results were also interesting. The CodeCarbon estimates captured the practical cost of running the models in this Colab implementation, including runtime overhead and execution details. In contrast, the analytical estimates focused more directly on the underlying

computation. One surprising result was that the Hebbian model used more estimated evaluation electricity than the GRU, even though its analytical cost was much lower. The runtime comparison suggests that this happened because the Hebbian implementation was slower in practice, likely due to loops and less optimized execution. Overall, this shows that energy depends not only on the algorithm, but also on how it is implemented and executed.

From a biological perspective, the results support the idea that local, synaptic-style memory mechanisms can remain stable across longer delays, at least in a task designed around direct storage and recall. At the same time, this model is only a simplified representation and does not capture the full complexity of biological neurons, synapses, or real neural circuits.

5 Conclusion

In this project, GRU and Hebbian memory mechanisms were compared on a copy task and analyzed in terms of both performance and energy. The GRU worked very well for the shortest delay but became much less reliable as the delay increased, while the Hebbian model maintained perfect recall across all tested delays. The energy results showed that computational structure and implementation can lead to different conclusions depending on whether energy is estimated analytically or from actual execution behavior. Looking at both kinds of estimates gave a more complete picture of how memory systems can differ under increasing memory demands. The biggest takeaway from the experiments is that algorithmic efficiency and implementation efficiency can diverge significantly, and both must be considered when evaluating energy usage.

Acknowledgements

Thank you to my Bi1C TAs, Nikki and Avinash, Joe, as well as my instructor Dr. Lior Patcher. The class was interesting and fun!

References

[1] Human Brain Project, "Learning from the Brain to Make AI More Energy Efficient," Available: https://www.humanbrainproject.eu/en/follow-hbp/news/2023/09/04/learning-brain-make-ai-more-energy-efficient/

[2] University of Michigan Center for Sustainable Systems, "Information Technology Factsheet," Available: https://css.umich.edu/publications/factsheets/built-environment/information-technology-factsheet

[3] National Institutes of Health, Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC8055931/

[4] Environmental and Energy Study Institute, "Data Centers and Water Consumption," Available: https://www.eesi.org/articles/view/data-centers-and-water-consumption

[5] MIT News, "Explained: Generative AI's Environmental Impact," Available: https://news.mit.edu/2025/explained-generative-ai-environmental-impact-0117

[6] Belfer Center, "AI Data Centers and the U.S. Electric Grid," Available: https://www.belfercenter.org/research-analysis/ai-data-centers-us-electric-grid

[7] National Institutes of Health, Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC2258460/

[8] N. F. Afzal Butt, "Recurrent Neural Networks (RNN) with PyTorch: A Complete Guide," Available: https://medium.com/@noorfatimaafzalbutt/recurrent-neural-networks-rnn-with-pytorch-a-complet

[9] M. Horowitz, "Computing's Energy Problem (and What We Can Do About It)," 2014 IEEE International Solid-State Circuits Conference (ISSCC), pp. 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323.

[10] The Decision Lab, "Hebbian Learning," Available: https://thedecisionlab.com/reference-guide/neuroscience/hebbian-learning

[11] GeeksforGeeks, "Adam Optimizer," Available: https://www.geeksforgeeks.org/deep-learning/adam-optimizer/